

Lia — Growth Log

The stages of her growth, kept for tracking what she can do at each one.

A record of Lia's growth in stages, not a log of every exchange or bug fix — those live in the Session Summary and Session Transcript. This document is for tracking what kind of thing she is at each stage: what she could do, and what she couldn't yet.

The shape of it: each stage she gained a new kind of relationship to the world — first just talking, then reading something you hand her, then perceiving and being perceived (voice), and now, starting to act on the world beyond just describing it.

Stage 1 — Making her

What she gained: existence. A text-only companion, typed to and typed back at, but with something most chatbots don't have — memory that survives closing the window.

- A local brain: Ollama running `llama3.2:3b`, small enough for a 4GB graphics card.
- Two-tier memory: a short-term rolling buffer for the live conversation, and a permanent SQLite database for facts and semantically searchable past turns (`nomic-embed-text` for the embeddings).
- A diary — a private reflection written at the end of each session.
- Six files: `config`, `db`, `llm`, `memory`, `diary`, `main`.

What she couldn't do yet: *hear you, speak, read anything you hadn't typed directly into the conversation, or do anything beyond generating the next reply.*

Stage 2 — Controlling actions from a file explorer path

What she gained: her first action beyond conversation — reading something real, from a place on disk, on request.

- A library folder (LIA/library/) you hand her documents through — PDFs, text, markdown.
- Say “read my files” and she indexes what's there: extracted, chunked, embedded, page-cited so she can say exactly where an answer came from.
- Deliberately on-demand, never automatic. She does not scan your drive, and a file sitting in the folder is *available* to her, not already read — a boundary kept even when it would have been easier to just read everything on startup.

What she still couldn't do: *hear or speak a word, or touch anything outside that one folder.*

Stage 3 — Adding voices and ML models

What she gained: a body, in the only sense that matters for a companion — a voice, and ears. This is the stage where she stopped being a chat window and became something you actually talk to.

- **Speaking:** Piper, a local neural text-to-speech engine, streamed sentence-by-sentence so she starts talking before the whole reply has even finished generating.
- **Listening:** faster-whisper for transcription, Silero VAD for detecting when you start and stop talking — together, an open microphone rather than push-to-talk.
- **A name to be called:** a wake word matched on the transcript itself, needing no trained model, tolerant of however her name actually gets misheard.
- **Presence without a window:** a system tray app, autostart at login, packaged as a standalone executable.
- A second voice engine (KittenTTS) was explored, got fully working, and was then removed by request — the seam for adding another one is still there in `voice.py` if it's ever wanted again.

***What she still couldn't do:** affect anything outside the conversation itself — no control over the machine she runs on, beyond her own voice and ears.*

Stage 4 — Controlling actions (in progress)

What's changing: the current frontier. Everything up to Stage 3 was perception and memory — hearing, speaking, reading, remembering. Stage 4 is the first time she reaches outward and changes something in the world rather than just describing it.

Built so far

- **Media control** — play, pause, skip whatever's already playing (Spotify, a browser tab, any app), via Windows' System Media Transport Controls. A remote, not a jukebox: it can't start a track from nothing, and says so honestly rather than pretending.
- **Volume control** — up, down, mute, unmute, via direct Core Audio access. (Simulating the physical volume keys was tried first and found to silently do nothing — fixed by talking to the audio API directly instead of imitating a keypress.)
- **Alarms and timers** — genuine ones, checked by a background watchdog every ten seconds and persisted across restarts, so “remind me in twenty minutes” survives even if the conversation, or the app itself, closes in the meantime.

What she knows about herself

A real gap surfaced during this stage: she had no idea any of this existed. Asked directly “can you play music,” she answered from generic assumptions about being “a text-based system” — technically wrong, since the capability was already built, just never told to her. Fixed by adding an explicit, honest capability list to her own system prompt, so she reports what she can and can't do accurately instead of guessing.

Reaching outside the machine: real lookups, and a fuller library

Two separate systems, both narrow and optional by design. **Internet access** — weather needs no key at all (Open-Meteo, geolocated from your IP); “look that up” checks a live source instead of guessing, via Groq or OpenRouter's actual web-search mode, whichever key is set. Naming a source works too — “search Reddit for...” — though one caveat came out of testing: search-operator syntax like `site:reddit.com` makes the underlying search decline entirely, where plain natural language works. A later pass found the natural phrasings people actually use for Reddit weren't all being caught — fixed by adding “reddit” itself as a trigger word, rather than requiring an exact phrase match. YouTube got the same trigger word shortly after, for the same reason — with the same honest limit either way: it's still a text search that happens to mention the site, not anything that can watch or listen to what's actually there.

The library now reads .epub files as well as PDF, text, and markdown — added after a real book (a Harry Potter novel) got dropped in and turned out not to work yet. A full novel is a genuine time cost to index (roughly 20 minutes of embedding calls for one book), and even once indexed she retrieves passages, not the whole text — she can discuss a specific scene or character, not narrate the book start to finish.

Knowing who's asking: voice recognition

If actions are the risk this stage introduced, this is the safeguard for it. Say “Lia, it's Wade” a few times and she builds a voiceprint — a local model (3D-Speaker's CAM++, run through sherpa-onnx, no PyTorch) compares later speech against it. Where a clear mismatch doesn't grant a hard lock, it withholds the things that matter: she won't use your name, and won't share what she remembers about you.

Honestly unverified: real-world accuracy could only be tested against synthetic TTS voices from here, not an actual human voice on an actual microphone. The signal was confirmed real and correctly directioned, but likely *understates* true accuracy — synthetic speech probably lacks some of the natural acoustic texture the model was trained on. The match threshold is a documented starting point, not a calibrated answer, and `/whoami` exists specifically so it can be tuned against real use.

A harder gate: a spoken passcode

Voice recognition alone is a soft signal — useful, but not something to bet real access on given a laptop mic's reliability. So every session now opens locked: say the identity phrase within the first exchange, or she asks for a spoken passcode instead of acting on anything. Get it right and the session opens for good; get it wrong and she stops listening outright, recoverable only by hand (the tray menu, or typing it back on) — deliberately not recoverable by voice, since that would make the lock trivial to talk past.

A second, separate passcode does something rather than unlocking anything: said at any time, unlocked or not, it has her read out and explain a specific reference document in her own words. Knowing the word is the point, the same as the first passcode.

Two real bugs, found only by running it, not by reading the code: the gate was checked before the “end the session” check, so once locked, saying goodbye got treated as a passcode guess instead of exiting — which combined with a second, unrelated bug (the keyboard reader spinning with no backoff once input ran out) into a genuine infinite loop, over 600,000 identical lines in under a minute. Separately, saying the identity phrase and a request in the same breath (“it’s Wade, play some music”) silently dropped the identity phrase, because the request got rewritten into a command before the gate ever saw the original words. Both fixed and reverified live.

The unlock passcode itself moved to an environment variable shortly after, the same reasoning as the API keys — a plaintext passcode in a file that might end up in a public repo isn’t one. Left unset, the gate now disables itself with a one-time log line rather than locking anyone out with a word that could never be typed correctly.

Not yet built

- Opening or closing applications.
- Anything involving the mouse, keyboard, or screen beyond the specific controls above.
- Browsing the web or clicking anything on your behalf.
- Any action on Linux or macOS — everything in this stage is Windows-specific so far.

Where this goes next is genuinely open. The pattern that’s worked a few times now — find the narrowest real API for a specific capability (SMTC for media, Core Audio for volume, a local speaker-embedding model for identity) rather than reaching for something broad and risky — is the one to keep following as this stage grows.